

ANÁLISIS DE PATRONES DE DISEÑO Y ARQUETIPOS

Proyecto: S.I.G.I. - Municipalidad Valle del Sol

Integrant

1. Introducción

Acá describimos los patrones de diseño y arquetipos que usamos en S.I.G.I. No los pusimos "por

catálogo'

Cubrimos backend (microservicios Spring) y frontend (React); la entrega es full stack.

2. Resumen ejecutivo

Categoría | Patrones / arquetipos principales

Arquitect

3. Arquetipos arquitectónicos

3.1 Estilo: microservicios

Qué es: El sistema se divide en servicios desplegados por separado, cada uno con

responsa

Por qué lo elegimos:

- El caso semestral y la asignatura de arquitectura pedían demostrar fronteras claras entre

módulos.

Trade-off que asumimos: más complejidad operativa (Eureka, Docker Compose, varios puertos).

Para el ta

flowchart TB

subgrap

GW --> R

GW --> UB

GW --> E

GW --> EMP

GW --> M

3.2 API Gateway (fachada de entrada)

Qué es: Un único punto HTTP (:8080) que enruta hacia los microservicios internos.

Por qué:

- El navegador no necesita conocer diez puertos distintos.

- Centrali

Implementación: api-gateway con Spring Cloud Gateway y filtro JwtAuthGatewayFilterFactory.

3.3 Service Discovery (Eureka)

Qué es: Los servicios se registran al arrancar; el Gateway resuelve lb://servicio-reporte

dinámica

Por qué: En Docker las IPs internas cambian. Eureka evita hardcodear hosts en el YAML del

Gateway

3.4 Database per Service

Qué es: Cada microservicio persiste en su propia base lógica (db_usuario, db_reporte,

db_empl

Por qué: Respeta el principio de datos dueños del servicio. En local usamos una sola instancia

MySQL c

3.5 Arquetipo de capas (por microservicio)

Dentro de cada servicio Spring repetimos:

Capa | Responsabilidad | Ejemplo

Controlle

Por qué: Es el arquetipo que más domina el equipo con Spring Boot; separa "cómo llega el JSON"

de "qué h

3.6 SPA + backend desacoplado (frontend)

Qué es: Sigi_Front es una Single Page Application en React que consume REST vía Gateway.

Por qué: Permite roles distintos en la misma app cambiando rutas y menú según JWT, sin

recargar

4. Patrones de diseño - Backend

4.1 Singleton (implícito en Spring)

Dónde: Beans anotados con `@Service`, `@Component`, `@Repository`.

Qué hace: Spring crea una instancia por contexto de aplicación y la reutiliza (inyección de

dependen

Justificación: Evita instanciar manualmente servicios y facilita testing con `@MockBean`.

Ejemplo en código: `UsuarioService`, `ReporteService`, `MediaService`.

4.2 Repository (GoF + Spring Data JPA)

Dónde: Interfaces que extienden `JpaRepository<Entity, Long>`.

Qué hace: Abstrae SQL/CRUD; métodos como `findByUsuarioldOrderByFechaReporteDesc` se generan por

convenci

Justificación: No mezclamos consultas SQL en los controladores. Cumple el principio de única

responsa

Ejemplo: `ReporteRepository`, `PostulacionRepository`, `MediaArchivoRepository`.

4.3 DTO (Data Transfer Object)

Dónde: Paquetes `dto` en cada microservicio (`ReporteResponse`, `UsuarioRegistroDTO`, `MediaResponse`).

Qué hace: Transporta solo los campos que el cliente debe ver; nunca enviamos la contraseña

hashead

Justificación:

- Desacopla el modelo de persistencia (JPA) del contrato REST.

- Permite

Ejemplo: `UsuarioResponseDTO.fromEntity(Usuario)` - método estático de mapeo que usamos en lugar

de un ma

4.4 Factory Method - Gateway Filter Factory

Dónde: `JwtAuthGatewayFilterFactory` extends `AbstractGatewayFilterFactory`.

Qué hace: Spring Cloud Gateway registra el filtro bajo el nombre `JwtAuth` en `application.yml`

(conven

Justificación: Encapsula la creación del filtro de autenticación; el YAML solo declara -

JwtAuth s

4.5 Facade (API Gateway)

Dónde: Módulo api-gateway.

Qué hace: Presenta una interfaz simple (/api/reportes, /auth/login) ocultando la topología

interna d

Justificación: Reduce acoplamiento del cliente respecto a puertos 8081-8088.

4.6 Proxy / Cliente declarativo - OpenFeign

Dónde:

- UbicacionClient en servicio-reporte

- Emergen

Qué hace: Interface Java con anotaciones; Spring genera el cliente HTTP hacia otro

microserv

Justificación: Evita escribir RestTemplate repetitivo y mantiene las llamadas entre servicios

tipadas.

4.7 Circuit Breaker (Resilience4j)

Dónde: UbicacionConsultaService.obtenerCoordenadas con @CircuitBreaker(name = "ubicacion",

fallbackM

Qué hace: Si el servicio de ubicación u OpenCage fallan repetidamente, ejecuta fallback y

devuelve

Justificación: Un reporte ciudadano no debe perderse porque la API externa de mapas no

respondi

sequenceDiagram

participa

4.8 Strategy (variante ligera - roles)

Dónde: Comprobaciones if ("ADMIN".equals(rol)) en controladores (EmpleoController,

ReporteC

Qué hace: Cambia el comportamiento del endpoint según el rol sin duplicar controladores

enteros.

Justificación: Para el alcance académico bastó con ramas explícitas; en producción

evaluaría

4.9 CommandLineRunner / Seed de datos

Dónde: UsuariosPruebaInitializer, DatosEjemploEmpleos, DatosEjemploValleDelSol (recursos).

Qué hace: Al iniciar el contexto, inserta datos demo si la tabla está vacía.

Justificación: Correcciones y demos con usuarios conocidos (Hawk, Emilio, Rodrigo) sin scripts

SQL man

4.10 Soft Delete

Dónde: UsuarioService.desactivarUsuario - marca activo = false en lugar de DELETE físico.

Justificación: Trazabilidad y posibilidad de auditoría municipal; patrón común en sistemas

administr

5. Patrones de diseño - Frontend (React)

5.1 Componentización (Composite)

Dónde: ReporteIncendioCard, Layout, Spinner, MapaIncidentes.

Qué hace: Divide la UI en piezas reutilizables con props.

Justificación: Trabajo de Full Stack III (desafío Municipalidad) y mantenibilidad; evita

duplicar l

5.2 Container / Presentational (flexible)

Dónde:

- Contenedores: páginas en src/pages/ (Dashboard, MisReportes) - cargan datos, manejan

useEffect

Justificación: Separa lógica de fetching de la visual; facilita tests de tarjetas sin mockear

toda la A

5.3 Provider (Context API)

Dónde: AuthProvider + hook useAuth.

Qué hace: Comparte sesión (token, rol, usuarioid) en todo el árbol sin prop drilling.

Justificación: Casi todas las rutas protegidas necesitan saber quién está logueado.

5.4 Módulo de servicios (capa de acceso a datos)

Dónde: src/services/apiClient.js, authService.js, reporteService.js, empleoService.js,

mediaSe

Qué hace: Centraliza URLs y formato de peticiones (JSON vs multipart).

Justificación: Si cambia el Gateway o un path, se toca un solo archivo. apiUpload existe

porque s

5.5 Patrón loading / error / data (async UI)

Dónde: MisReportes, Empleos, Dashboard, Emergencias.

Qué hace:

loading === true -> Spinner

error !==

Justificación: Requisito explícito del curso Full Stack III; evita pantallas en blanco cuando

el backen

5.6 Estado derivado (sin useEffect innecesario)

Dónde: reportesFiltrados = reportes.filter(...) en Reportes.jsx; contadores en dashboard.

Qué hace: Calcula listas filtradas en render a partir de estado existente.

Justificación: El profesor enfatizó no duplicar estado que se puede derivar; menos bugs de

sincroniz

5.7 Mapper / Adapter

Dónde: reporteApiACard en reporteMappers.js.

Qué hace: Adapta respuesta API (prioridad: CRITICA) al formato de tarjeta del trabajo de

clases (n

Justificación: Reutilizamos el componente del desafío sin mentir con datos hardcodeados cuando

hay API r

5.8 Protected Route (guard de navegación)

Dónde: ProtectedRoute.jsx + rutas en App.jsx con roles={[...]}.

Qué hace: Redirige a login si no hay token; bloquea rutas admin a ciudadanos.

Justificación: Seguridad UX alineada con roles del JWT (la seguridad real sigue en el Gateway).

6. Tabla de trazabilidad: problema -> patrón

7. Patrones considerados y descartados

Patrón | Motivo de descarte

CQRS co

8. Conclusión del análisis

Combinamos microservicios con patrones que ya conocemos en Spring (Repository, DTO, Feign,

Circuit Br

Para la defensa oral nos quedamos con tres ejemplos en vivo:

1. JWT en Gateway -> cabeceras hacia reportes.

2. Circuit

Elaborado por Hawk Durant, Emilio Jaramillo y Rodrigo Candia - S.I.G.I., Mayo 2026.